

RAPPORT DE PROJET DE MATHÉMATIQUES

Méthodes itératives pour le Laplacien

Étudiants

Manh Hung	NGUYEN	mnguy00@insa-rouen.fr
Tan Minh Duy	NGO	tn00@insa-rouen.fr

13 décembre 2025

Encadré par

Antoine TONNOIR

Synthèse du projet

Titre du projet : Résolution efficace du Laplacien multidimensionnel par méthodes itératives : analyse des performances de Jacobi, Gradient, CG et PCG sur matrices creuses au format CSR

Résumé : Dans ce projet, nous étudions la résolution numérique de l'équation du Laplacien en dimensions 1D, 2D et 3D. Après une discrétisation par différences finies, le problème continu est ramené à la résolution d'un système linéaire de grande taille, stocké sous forme creuse au format CSR (Compressed Sparse Row). Plusieurs méthodes itératives sont implémentées et comparées : la méthode de Jacobi, la descente de gradient, le gradient conjugué (CG) et le gradient conjugué préconditionné (PCG). L'analyse porte sur la convergence des méthodes, le nombre d'itérations, le coût en temps de calcul et l'influence de la dimension du problème. Les résultats numériques montrent que les méthodes classiques deviennent rapidement inefficaces lorsque la taille du système augmente. À l'inverse, les méthodes de Krylov, en particulier le gradient conjugué préconditionné par SSOR, offrent des performances nettement supérieures. Ce travail met ainsi en évidence l'importance du choix de la méthode numérique et du préconditionnement pour la résolution efficace de systèmes linéaires creux issus d'équations aux dérivées partielles.

Mots clés : Laplacien ; méthodes itératives ; Analyse numérique ; CSR ; gradient conjugué ; préconditionnement

Table des matières

Synthèse du projet	1
Introduction	3
1 Modélisation mathématique du problème	4
1.1 Problème continu	4
1.2 Schéma aux différences finies	5
1.3 Système linéaire obtenu	7
1.4 Propriétés de la matrice A	10
2 Méthodes de résolution itérative	11
2.1 Méthode de Jacobi	11
2.2 Méthode du Gradient Descent	13
2.3 Méthode du Gradient Conjugué	15
2.4 Méthode du Gradient Conjugué Préconditionné	18
3 Résultats numériques et comparaison des méthodes	21
3.1 Cadre numérique et implémentation	21
3.2 Résultats en dimension 1D	22
3.3 Résultats en dimension 2D	25
3.4 Résultats en dimension 3D	27
Conclusion	30

Introduction

La résolution numérique de l'équation du Laplacien occupe une place centrale dans de nombreux domaines de la physique et de l'ingénierie. Elle intervient notamment en électromagnétisme, en mécanique des fluides, en transfert thermique ou encore dans l'étude des champs gravitationnels. Dans ces contextes, le problème revient souvent à calculer le potentiel d'un champ à partir d'une équation aux dérivées partielles, puis à en déduire la grandeur physique correspondante, comme le champ électrique ou magnétique. Cependant, la discrétisation du Laplacien conduit à des systèmes linéaires de grande taille, dont la résolution efficace représente un véritable défi numérique.

Dans ce projet, nous nous intéressons à différentes méthodes itératives pour résoudre ces systèmes, issues de la discrétisation par différences finies en une, deux et trois dimensions. Ce type de discrétisation produit des matrices creuses, ce qui rend l'utilisation de techniques adaptées indispensable. Nous travaillons ainsi avec le format CSR (Compressed Sparse Row), qui permet de réduire considérablement le coût en mémoire et d'accélérer les produits matrice-vecteur, opérations essentielles dans les algorithmes itératifs.

L'objectif principal de ce travail est de comparer plusieurs méthodes numériques : Jacobi, descente de gradient, gradient conjugué et gradient conjugué préconditionné. Chacune de ces méthodes présente des propriétés spécifiques en termes de rapidité de convergence, de stabilité et de coût de calcul. L'analyse de leurs performances sur des problèmes de plus en plus dimensionnels permet d'identifier les approches les plus adaptées aux grands systèmes issus de la simulation physique.

Enfin, nous appliquons ces techniques à un cas concret lié à l'électrostatique. En résolvant le Laplacien en trois dimensions, nous reconstruisons le potentiel électrique et visualisons les lignes de champ associées.

1 Modélisation mathématique du problème

1.1 Problème continu

L'objectif de ce travail est de résoudre numériquement une équation du type Poisson ou Laplace, qui apparaît dans de nombreux phénomènes physiques. Dans sa forme générale, le problème consiste à déterminer une fonction u définie sur un domaine $[0, 1]^d$ telle que :

$$-\Delta u + k u = f \quad \text{dans } [0, 1]^d,$$

où d représente la dimension du problème ($d = 1, 2$ ou 3), Δ est l'opérateur Laplacien, $k \geq 0$ est un paramètre réel, et f désigne un terme source. Dans de nombreuses applications physiques, ce terme source modélise par exemple la présence d'une charge électrique ou d'un aimant. Le terme ku peut également être interprété comme une contribution de réaction ou d'absorption selon le contexte.

Ce problème est complété par des conditions aux limites de type Dirichlet. Dans notre étude, nous imposons que la fonction u soit nulle sur le bord du domaine :

$$u = 0 \quad \text{sur } \partial([0, 1]^d).$$

En pratique, cela signifie que le potentiel est fixé à zéro sur toutes les faces du domaine, ce qui correspond à une modélisation fréquente en électrostatique. Dans le cas particulier de la dimension un, les conditions deviennent simplement :

$$u(0) = 1, \quad u(1) = 0.$$

La résolution analytique de ce type d'équation n'est généralement possible que dans des cas simples. Dès que le domaine, le terme source ou les paramètres physiques deviennent complexes, une résolution numérique est nécessaire. C'est pourquoi nous cherchons à remplacer l'équation continue par un modèle discret, basé sur l'approximation de la dérivée seconde et du Laplacien. Ce passage au modèle discret nous conduira à un système linéaire de grande taille qu'il faudra résoudre efficacement à l'aide de méthodes itératives.

1.2 Schéma aux différences finies

Pour résoudre numériquement le problème continu présenté dans la section précédente, nous remplaçons les dérivées par des approximations discrètes. L'idée principale est de remplacer le domaine continu $[0, 1]^d$ par un ensemble fini de points, appelés noeuds de discrétisation, et d'approximer l'opérateur de dérivation par des formules faisant intervenir les valeurs de la solution en ces noeuds.

Discrétisation en dimension 1

En dimension un, on considère le segment $[0, 1]$ et on le découpe en $n + 1$ intervalles de même longueur. On définit le pas de discrétisation

$$h = \frac{1}{n + 1},$$

et les points du maillage

$$x_i = i h, \quad \text{pour } i = 0, 1, \dots, n + 1.$$

La solution inconnue $u(x)$ est alors approchée par une suite de valeurs $u_i \approx u(x_i)$. Les conditions aux limites sont imposées en fixant $u_0 = 1$ et $u_{n+1} = 0$ dans le cas 1D du TP.

Pour approcher la dérivée seconde $u''(x)$, nous utilisons le schéma classique à trois points, d'ordre deux, donné par :

$$u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}, \quad \text{pour } i = 1, \dots, n.$$

Et on obtient, pour chaque point intérieur x_i :

$$-\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} + k u_i = f(x_i).$$

Cette relation fournit une équation linéaire reliant les valeurs u_{i-1} , u_i et u_{i+1} . En regroupant ces équations pour tous les points intérieurs, nous construisons un système linéaire pour les inconnues $(u_1, \dots, u_n)$. Les valeurs aux bords u_0 et u_{n+1} sont déjà connues grâce aux conditions de Dirichlet et sont donc déplacées dans le second membre.

Extensions en dimension 2 et 3

En dimension deux ou trois, le principe reste le même, mais le Laplacien agit maintenant sur plusieurs variables. En dimension deux, par exemple, pour une fonction $u(x, y)$, on a :

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}.$$

On introduit un maillage régulier du carré $[0, 1]^2$ en choisissant le même pas $h = \frac{1}{n+1}$ dans les deux directions, et on définit les points

$$x_i = i h, \quad y_j = j h, \quad \text{pour } i, j = 0, \dots, n+1.$$

La valeur de la solution au point (x_i, y_j) est notée $u_{i,j} \approx u(x_i, y_j)$. Pour les points intérieurs, la dérivée seconde selon x est approchée par

$$\frac{\partial^2 u}{\partial x^2}(x_i, y_j) \approx \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h^2},$$

et de même pour la dérivée seconde selon y :

$$\frac{\partial^2 u}{\partial y^2}(x_i, y_j) \approx \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h^2}.$$

En combinant ces deux approximations, on obtient un schéma à cinq points pour le Laplacien :

$$-\Delta u(x_i, y_j) \approx -\frac{u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{i,j}}{h^2}.$$

On obtient pour tout point intérieur (x_i, y_j) avec $1 \leq i, j \leq n$:

$$-\frac{u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{i,j}}{h^2} + k u_{i,j} = f(x_i, y_j).$$

Cette équation relie la valeur inconnue $u_{i,j}$ à ses quatre voisins directs. L'ensemble de ces équations pour tous les points intérieurs du maillage constitue un système linéaire dont les inconnues sont les composantes de u sur les points du carré.

Pour le cas tridimensionnel, la logique de la discrétisation reste exactement la même en dimensions trois, mais chaque point intérieur possède désormais des voisins dans une dimension supplémentaire. On introduit une nouvelle direction z définie par

$$z_k = k h, \quad \text{pour } k = 0, \dots, n+1,$$

et la fonction inconnue est approchée par $u_{i,j,k} \approx u(x_i, y_j, z_k)$. Le Laplacien en 3D s'écrit

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2},$$

En appliquant la discrétisation en différences finies sur les trois directions, on obtient pour tout point intérieur (x_i, y_j, z_k) , avec $1 \leq i, j, k \leq n$:

$$-\frac{u_{i-1,j,k} + u_{i+1,j,k} + u_{i,j-1,k} + u_{i,j+1,k} + u_{i,j,k-1} + u_{i,j,k+1} - 6u_{i,j,k}}{h^2} + k u_{i,j,k} = f(x_i, y_j, z_k).$$

Dans ce schéma à sept points, la valeur en chaque noeud dépend donc des six voisins directs dans les directions $\pm x$, $\pm y$ et $\pm z$. Comme en dimension deux, la condition de Dirichlet impose que $u = 0$ sur toutes les faces du cube.

1.3 Système linéaire obtenu

La discrétisation par différences finies transforme l'équation différentielle du problème continu en un ensemble d'équations algébriques portant sur les valeurs de u aux points intérieurs du maillage. L'ensemble de ces équations peut être rassemblé sous la forme matricielle

$$A u = b,$$

où A est la matrice du système linéaire, u est le vecteur des inconnues et b regroupe le terme source ainsi que les contributions des conditions aux limites. La construction de ces trois objets dépend de la dimension du problème. Nous détaillons successivement les cas 1D, 2D et 3D.

Cas 1D

Le domaine $n + 2$ points avec un pas

$$h = \frac{1}{n + 1}.$$

En notant $u_i \approx u(x_i)$ et en appliquant le schéma à trois points, l'équation discrète au point intérieur x_i ($i = 1, \dots, n$) devient :

$$\left(\frac{2}{h^2} + k\right) u_i - \frac{1}{h^2} u_{i-1} - \frac{1}{h^2} u_{i+1} = f(x_i) + \text{contributions des conditions de Dirichlet}.$$

En regroupant les n inconnues dans le vecteur

$$u = (u_1, \dots, u_n)^T,$$

on obtient un système linéaire $Au = b$ où la matrice A est tridiagonale :

$$A = \begin{pmatrix} a & c & 0 & \cdots & 0 \\ c & a & c & \ddots & \vdots \\ 0 & c & a & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & c \\ 0 & \cdots & 0 & c & a \end{pmatrix},$$

avec

$$a = \frac{2}{h^2} + k, \quad c = -\frac{1}{h^2}.$$

Cas 2D

Dans le cas bidimensionnel, le domaine $[0, 1]^2$ est discrétisé en un maillage régulier de $n \times n$ points intérieurs. Pour chaque coupe (i, j) , on a :

$$u_{i,j} \approx u(x_i, y_j), \quad 1 \leq i, j \leq n.$$

L'approximation du Laplacien à l'aide du schéma à cinq points relie la valeur $u_{i,j}$ à ses quatre voisins directs :

- deux voisins dans la direction x : $u_{i-1,j}$ et $u_{i+1,j}$,
- deux voisins dans la direction y : $u_{i,j-1}$ et $u_{i,j+1}$.

L'équation discrète associée au point intérieur (i, j) devient :

$$\left(\frac{4}{h^2} + k\right) u_{i,j} - \frac{1}{h^2} u_{i-1,j} - \frac{1}{h^2} u_{i+1,j} - \frac{1}{h^2} u_{i,j-1} - \frac{1}{h^2} u_{i,j+1} = f_{i,j} + \text{contributions des bords}.$$

Pour obtenir une formation matricielle, toutes les valeurs $u_{i,j}$ doivent être rangées dans un seul vecteur. Pour cela, on associe à chaque coupe (i, j) un indice :

$$\ell = (j-1)n + i, \quad 1 \leq i, j \leq n, \quad \text{et} \quad u_\ell = u_{i,j}.$$

La ligne ℓ de la matrice A contient alors cinq coefficients non nuls :

$$\left(\frac{4}{h^2} + k\right) u_\ell - \frac{1}{h^2} u_{\ell-1} - \frac{1}{h^2} u_{\ell+1} - \frac{1}{h^2} u_{\ell-n} - \frac{1}{h^2} u_{\ell+n} = b_\ell.$$

Ainsi, la matrice A en 2D n'est plus simplement tridiagonale comme en 1D, elle contient maintenant des blocs tridiagonaux représentant les interactions dans la direction x , et des blocs $-I$ pour les interactions dans la direction y :

$$A = \begin{pmatrix} T & -I & 0 & \cdots & 0 \\ -I & T & -I & \ddots & \vdots \\ 0 & -I & T & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -I \\ 0 & \cdots & 0 & -I & T \end{pmatrix}.$$

Cas 3D

Dans le cube $[0, 1]^3$, les inconnues sont $u_{i,j,k} \approx u(x_i, y_j, z_k)$ pour $1 \leq i, j, k \leq n$. Le schéma à sept points donne :

$$\left(\frac{6}{h^2} + k\right) u_{i,j,k} - \frac{1}{h^2} u_{i-1,j,k} - \frac{1}{h^2} u_{i+1,j,k} - \frac{1}{h^2} u_{i,j-1,k} - \frac{1}{h^2} u_{i,j+1,k} - \frac{1}{h^2} u_{i,j,k-1} - \frac{1}{h^2} u_{i,j,k+1} = f_{i,j,k} + \text{tdb}.$$

Pour linéariser les inconnues, on utilise maintenant :

$$\ell = (k-1)n^2 + (j-1)n + i, \quad u_\ell = u_{i,j,k}.$$

Chaque équation du système relie u_ℓ à six voisins :

$$\ell \pm 1, \quad \ell \pm n, \quad \ell \pm n^2.$$

La matrice A possède donc sept coefficients non nuls par ligne. Sa structure générale est obtenue en empilant des blocs 2D :

$$A = \begin{pmatrix} B & -C & 0 & \cdots & 0 \\ -C & B & -C & \ddots & \vdots \\ 0 & -C & B & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -C \\ 0 & \cdots & 0 & -C & B \end{pmatrix},$$

où :

- B est la matrice bloc du cas 2D,
- $-C$ représente les interactions entre plans successifs selon z .

1.4 Propriétés de la matrice A

La compréhension des propriétés de la matrice A est essentielle pour choisir des méthodes numériques de résolution adaptées. La matrice A provient de la discrétisation de l'opérateur $-\Delta u + k$ et possède plusieurs caractéristiques importantes.

Matrice creuse (sparse) Seuls les voisins directs d'un point du maillage interviennent dans l'équation discrète. Par conséquent, chaque ligne de A contient très peu de coefficients non nuls et la majorité des éléments de A sont nuls. Cette structure creuse devient de plus en plus marquée lorsque la taille du maillage augmente. Cette propriété justifie l'utilisation de formats de stockage adaptés, comme le format **CSR (Compressed Sparse Row)**, afin de réduire le coût mémoire et d'accélérer les produits matrice-vecteur.

Matrice symétrique Les coefficients reliant un point à ses voisins sont identiques dans les deux directions. Par exemple, en 2D, le coefficient associé à la relation entre $u_{i,j}$ et $u_{i+1,j}$ est le même que celui entre $u_{i+1,j}$ et $u_{i,j}$. Ainsi :

$$A_{p,q} = A_{q,p} \quad \text{pour tous } p, q.$$

Matrice définie positive Lorsque le paramètre k est positif ou nul,

$$u^T A u > 0 \quad \text{pour tout vecteur } u \neq 0.$$

Cela signifie que A est définie positive. Cette propriété garantit l'existence et l'unicité de la solution du système linéaire et la convergence des méthodes itératives telles que le gradient conjugué.

2 Méthodes de résolution itérative

2.1 Méthode de Jacobi

La méthode de Jacobi est l'une des méthodes itératives classiques pour la résolution de systèmes linéaires de grande taille. Elle repose sur l'idée d'améliorer progressivement une approximation de la solution sans factoriser la matrice A . Cette approche est particulièrement adaptée aux matrices creuses issues de la discrétisation du Laplacien.

On considère le système

$$Au = b.$$

La méthode de Jacobi s'appuie sur la décomposition (splitting) de la matrice A :

$$A = M - N,$$

où M doit être facile à inverser. Dans Jacobi, on choisit :

$$M = D = \text{Diag}(A) \text{ et } N = M - A.$$

En remplaçant dans le système,

$$Du = b - Nu,$$

ce qui conduit au schéma itératif :

$$u^{k+1} = D^{-1}(b - Nu^k).$$

Comme $N = A - D = L + U$, on obtient la formule composante par composante :

$$u_i^{(k+1)} = \frac{1}{A_{ii}} \left(b_i - \sum_{j \neq i} A_{ij} u_j^{(k)} \right).$$

Voici l'algorithme itératif de Jacobi :

Algorithm 1 Méthode de Jacobi

Require: $A, b, u_0, \maxIter, tol$

```
1:  $D \leftarrow \text{diag}(A)$ 
2:  $R \leftarrow A - D$ 
3:  $u \leftarrow u_0$ 
4: for  $k = 0$  to  $\maxIter - 1$  do
5:    $u_{\text{new}} \leftarrow D^{-1}(b - Ru)$ 
6:   if  $\|u_{\text{new}} - u\| < tol$  then
7:     return  $u_{\text{new}}$ 
8:   end if
9:    $u \leftarrow u_{\text{new}}$ 
10: end for
11: return  $u$ 
```

La méthode est bien définie si et seulement si $A_{ii} \neq 0$ pour tout i . D'après les résultats théoriques vues en cours, la méthode de Jacobi converge si la matrice d'itération

$$T = -D^{-1}(L + U)$$

vérifie

$$\rho(T) < 1,$$

où $\rho(T)$ est le rayon spectral de T .

Dans le cas du système issu de la discrétisation du Laplacien en 1D, on obtient :

$$A_{ii} = \frac{2}{h^2} + k, \quad A_{i,i\pm 1} = -\frac{1}{h^2}.$$

La condition de dominance diagonale conduit à :

$$\frac{2}{h^2} + k \geq \frac{2}{h^2} \iff k \geq 0.$$

Donc la méthode de Jacobi est applicable et converge pour toutes les valeurs $k \geq 0$. Cependant, lorsque $k = 0$ et que la dimension du problème augmente, la convergence peut devenir lente.

2.2 Méthode du Gradient Descent

La méthode de la descente de gradient est une méthode itérative permettant de résoudre le système linéaire

$$Ax = b$$

lorsque la matrice A est symétrique définie positive. L'idée fondamentale consiste à reformuler le problème de résolution du système comme un problème d'optimisation, puis à chercher la solution en minimisant une fonction énergie quadratique.

On introduit la fonction

$$J(x) = \frac{1}{2}(Ax, x) - (b, x),$$

où $(\cdot, \cdot)$ désigne le produit scalaire euclidien.

Le théorème suivant garantit le lien entre le système linéaire et l'optimisation :

Théorème. Si A est symétrique définie positive, alors la solution x^* du système $Ax = b$ est l'unique point qui minimise la fonction $J(x)$:

$$x^* = \arg \min_{x \in \mathbb{R}^n} J(x).$$

Le gradient de cette fonction vaut :

$$\nabla J(x) = Ax - b.$$

Ainsi, la condition d'optimalité $\nabla J(x^*) = 0$ est équivalente au système $Ax = b$.

Le gradient indique la direction de croissance maximale de la fonction. Pour atteindre le minimum, on se déplace dans la direction opposée au gradient. On obtient donc l'itération

$$x^{k+1} = x^k - \alpha(Ax^k - b),$$

où $\alpha > 0$ est le pas de descente.

Le résidu est défini par :

$$r^k = Ax^k - b = \nabla J(x^k),$$

ce qui permet de réécrire simplement :

$$x^{k+1} = x^k - \alpha r^k.$$

La méthode de descente peut être formulée comme une méthode de splitting

$$A = M - N.$$

Pour la descente de gradient à pas constant, on choisit :

$$M = \frac{1}{\alpha} I_d \implies M^{-1} = \alpha I_d,$$

$$N = M - A = \frac{1}{\alpha} (I_d - \alpha A).$$

Ainsi, la mise à jour

$$x^{k+1} = x^k - \alpha r^k$$

correspond bien à un splitting particulier.

L'algorithme est convergent si le pas α vérifie :

$$0 < \alpha < \frac{2}{\lambda_n},$$

où λ_n est la plus grande valeur propre de A .

Le pas optimal (celui qui minimise le rayon spectral de l'opérateur d'itération) est donné par :

$$\alpha_{\text{opt}} = \frac{2}{\lambda_n + \lambda_1},$$

où λ_1 est la plus petite valeur propre de A .

Cette méthode peut encore être améliorée en adaptant le pas α optimal à chaque itération. L'itération devient alors :

$$x^{k+1} = x^k - \alpha_k r^k.$$

On choisit α_k de façon à minimiser la fonction $J(x^{k+1})$, ce qui conduit à l'expression :

$$\alpha_k = \frac{\|r^k\|^2}{(Ar^k, r^k)}.$$

Algorithm 2 Gradient descent à pas variable

Require: A, b, x^0, ε

```
1:  $x \leftarrow x^0$ 
2:  $r \leftarrow Ax - b$ 
3: while  $\|r\| \geq \varepsilon\|r^0\|$  do
4:    $\alpha \leftarrow \frac{(r, r)}{(Ar, r)}$ 
5:    $x \leftarrow x - \alpha r$ 
6:    $r \leftarrow Ax - b$ 
7: end while
8: return  $x$ 
```

Bien que plus performant que Jacobi, la descente de gradient reste sensible au conditionnement de A . Dans les dimensions 2D et 3D, la convergence peut rester lente.

2.3 Méthode du Gradient Conjugué

La méthode du gradient conjugué (CG) est une amélioration importante de la descente de gradient pour résoudre le système linéaire

$$Ax = b,$$

où A est la matrice symétrique définie positive. Comme la descente de gradient, CG peut être vue à la fois comme une méthode d'optimisation et comme une méthode de Krylov. Elle utilise des directions de recherche spéciales, dites *A-conjuguées*, qui permettent une convergence beaucoup plus rapide.

On se place à partir d'une approximation initiale x_0 et on note le résidu initial :

$$r_0 = Ax_0 - b$$

L'espace de Krylov d'ordre p associé est défini par :

$$K(p) = \text{Vect}\{r_0, Ar_0, A^2r_0, \dots, A^{p-1}r_0\}.$$

On peut montrer que la suite des espaces $K(p)$ est croissante et qu'il existe un entier $p_{\max} \leq n$ tel que la solution exacte x^* du système vérifie :

$$x^* \in x_0 + K(p_{\max}).$$

Cela signifie qu'il est pertinent de chercher des approximations de la solution sous la forme

$$x_p \in x_0 + K(p),$$

et d'augmenter progressivement la dimension p . Une méthode de Krylov, comme CG, consiste précisément à construire itérativement de telles approximations x_p .

Comme pour la descente de gradient, on introduit la fonction quadratique

$$J(x) = \frac{1}{2}(Ax, x) - (b, x),$$

dont le minimum est atteint en x^* , solution de $Ax = b$.

Dans le cas d'une matrice symétrique définie positive, le produit scalaire

$$(x, y)_A = (Ax, y)$$

définit une norme $\|x\|_A$. On peut montrer l'équivalence des trois propriétés suivantes :

1. x^* est la solution de $Ax = b$;
2. $\tilde{J}(x) = \|x - x^*\|_A^2$ est minimale en x^* ;
3. x^* minimise $J(x)$.

Pour la méthode CG, on choisit comme critère d'optimalité :

$$x_p \in x_0 + K(p) \quad \text{tel que} \quad \|x_p - x^*\|_A = \min_{x \in x_0 + K(p)} \|x - x^*\|_A.$$

Autrement dit, à l'ordre p , l'approximation x_p est la meilleure possible (au sens de la norme A) dans l'espace affine $x_0 + K(p)$.

Pour simplifier ce problème de minimisation, on construit une base $(\omega_i)_{i=1,\dots,p}$ de $K(p)$ vérifiant la propriété de conjugaison A :

$$(A\omega_i, \omega_j) = 0 \quad \text{si } i \neq j.$$

En notant $W(p)$ la matrice dont les colonnes sont les vecteurs ω_i , cette propriété s'écrit matriciellement :

$$W(p)^T A W(p) = D(p),$$

où $D(p)$ est une matrice diagonale définie positive.

Dans cette base, le problème de minimisation sur $x_0 + K(p)$ se ramène à un problème simple sur $\mathbb{R}^p$, avec une matrice diagonale. On en déduit une formule explicite pour le

coefficient z_p utilisé dans la combinaison des directions, puis une récurrence courte du type :

$$x_{p+1} = x_p + z_{p+1} \omega_{p+1}.$$

Une propriété importante est que, dans cette base A -conjuguée, les composantes déjà calculées ne sont plus modifiées aux itérations suivantes, ce qui rend la méthode très efficace.

Algorithm 3 Algorithme du Gradient Conjugué

Require: A, b, x^0, ε

```

1:  $x \leftarrow x^0$ 
2:  $r^0 \leftarrow Ax - b$ 
3:  $w \leftarrow r^0$ 
4: while  $\|r\| \geq \varepsilon \|r^0\|$  do
5:    $z \leftarrow \frac{(r, w)}{(Aw, w)}$ 
6:    $x \leftarrow x - zw$ 
7:    $r \leftarrow r - zAw$ 
8:    $L \leftarrow \frac{(Ar, w)}{(Aw, w)}$ 
9:    $w \leftarrow r - Lw$ 
10: end while
11: return  $x$ 
```

Le gradient conjugué ressemble à la descente de gradient, mais il est bien supérieur en efficacité. En effet, la descente de gradient suit simplement le gradient (ou le résidu) à chaque itération, tandis que CG construit une suite de directions mutuellement A -conjuguées, ce qui évite de revenir en arrière et exploite mieux la structure de A SDP.

Un théorème classique (admis dans le cours) donne une estimation de l'erreur après k itérations :

$$\|x_k - x^*\|_2 \leq C \left(\frac{\sqrt{\text{cond}_2(A)} - 1}{\sqrt{\text{cond}_2(A)} + 1} \right)^k \|x_0 - x^*\|_2,$$

où $\text{cond}_2(A)$ est le conditionnement de A dans la norme 2.

Cette relation montre que : plus le conditionnement de A est mauvais (valeur propre la plus grande très supérieure à la plus petite) et plus le facteur

$$\frac{\sqrt{\text{cond}_2(A)} - 1}{\sqrt{\text{cond}_2(A)} + 1}$$

est proche de 1, plus la convergence est lente.

2.4 Méthode du Gradient Conjugué Préconditionné

Dans la section précédente, on voit clairement que la vitesse de convergence dépend fortement du conditionnement de la matrice A . Si $\text{cond}_2(A)$ est très grand, le facteur entre parenthèses est proche de 1 et la convergence devient lente. Pour améliorer cette situation, on introduit la notion de **préconditionnement**.

Le préconditionnement consiste à remplacer le système linéaire par un système équivalent mais mieux conditionné. On choisit une matrice M , appelée préconditionneur, telle que M est facile à inverser et la matrice préconditionnée MA a un conditionnement beaucoup plus petit que A . L'idée générale est donc de résoudre :

$$MAx = Mb,$$

ou bien, de manière équivalente, d'appliquer CG au système

$$Ax = b$$

mais en remplaçant le résidu r^k par un résidu préconditionné $z^k = M^{-1}r^k$.

Dans ce cas, la méthode de CG devient PCG (Preconditioned Conjugate Gradient). Théoriquement, le résultat de convergence s'écrit en remplaçant $\text{cond}_2(A)$ par $\text{cond}_2(MA)$, beaucoup plus favorable si le préconditionneur est bien choisi.

Le schéma de PCG est très proche de celui de CG, mais à chaque itération on doit résoudre un petit système avec M . Dans le code, c'est exactement le rôle de la fonction `precond(A, r, choix)` : elle renvoie $z = M^{-1}r$ pour un préconditionneur donné.

Par exemple, le cas `choix == 'D'` dans le code correspond au préconditionneur le plus simple :

$$M = D = \text{Diag}(A).$$

En effet, on divise chaque composante de r par l'élément diagonal correspondant $A_{\ell\ell}$, ce qui correspond à appliquer le préconditionneur de Jacobi.

Le cas le plus intéressant pour ce projet est `choix == 'SSOR'`. On part de la décomposition classique d'une matrice symétrique définie positive :

$$A = D - E - E^T,$$

où

- D est la matrice diagonale de A ,
- E contient la partie strictement inférieur ($-L$),
- E^T la partie strictement supérieur ($-L^T$).

Le préconditionneur **SSOR (Symmetric Successive Over-Relaxation)** est un préconditionneur qui combine deux balayages de type SOR, l'un en remontant et l'autre en descendant, de façon symétrique. Théoriquement, une forme classique de ce préconditionneur est :

$$M_{\text{SSOR}} = \frac{1}{2 - \omega} (D - \omega E) D^{-1} (D - \omega E^T),$$

avec un paramètre de relaxation $0 < \omega < 2$ (dans le code, $\omega = 1.77$).

Dans l'implémentation, on ne garde pas forcément le facteur scalaire $\frac{1}{2 - \omega}$, ce qui ne change pas la direction du préconditionnement, seulement l'échelle.

Explication du SSOR dans precond .

a) Vecteurs et stockage CSR

- `lig`, `col`, `val` décrivent la matrice A au format CSR (Compressed Sparse Row).
- `r` est le résidu d'entrée.
- `Cr` sert à stocker le vecteur transformé $z = M^{-1}\mathbf{r}$.

L'objectif de ce bloc est de calculer approximativement :

$$z = (D - \omega E)^{-1} D (D - \omega E^T)^{-1} \mathbf{r},$$

c'est-à-dire d'appliquer M_{SSOR}^{-1} au vecteur `r`.

b) Étape 1 – Remontée

On résout le système

$$(D - \omega E^T)y = r$$

par substitution arrière, et on met y dans `Cr`.

Boucle de `ll = nn-1` vers 0 :

- on parcourt les colonnes `cc` de la ligne `ll` :
- si `cc == ll`, on récupère l'élément diagonal A_{ll} dans `valdiag`;

— si $cc > ll$, on utilise ces coefficients pour mettre à jour :

$$Cr[ll] \leftarrow Cr[ll] - \omega A_{ll,cc} Cr[cc].$$

— ensuite on divise par `valdiag` :

$$Cr[ll] \leftarrow \frac{Cr[ll]}{A_{ll}}.$$

c) Étape 2 – Multiplication par la diagonale

Dans la boucle diagonale , on effectue :

$$Cr[ll] \leftarrow A_{ll} Cr[ll].$$

Cela correspond à la multiplication par D :

$$y \leftarrow D y.$$

d) Étape 3 - Descente

Cette étape réalise la substitution avant pour résoudre :

$$(D - \omega E)z = Dy.$$

Boucle de $ll = 0$ vers $nn-1$:

— pour chaque ligne ll , on parcourt les colonnes cc :

— si $cc < ll$, on met à jour :

$$Cr[ll] \leftarrow Cr[ll] - \omega A_{ll,cc} Cr[cc].$$

— puis on divise à nouveau par `valDiag[ll] = A_{ll}` :

$$Cr[ll] \leftarrow \frac{Cr[ll]}{A_{ll}}.$$

Au final, le vecteur renvoyé est bien :

$$Cr = z = (D - \omega E)^{-1} D (D - \omega E^T)^{-1} r,$$

c'est-à-dire

$$z = M_{\text{SSOR}}^{-1} r.$$

3 Résultats numériques et comparaison des méthodes

Dans ce chapitre, nous présentons les résultats numériques obtenus pour la résolution de l'équation du Laplacien en 1D, 2D et 3D. L'objectif est de comparer les différentes méthodes itératives étudiées, d'analyser leur convergence, leur coût numérique et de mettre en évidence l'apport du préconditionnement, en particulier dans les problèmes de grande dimension.

3.1 Cadre numérique et implémentation

Cette section précise le cadre expérimental utilisé pour produire les résultats présentés ensuite. L'objectif est d'assurer la reproductibilité des expériences et de clarifier le lien entre les méthodes théoriques (Chapitre 3) et leur mise en oeuvre.

Les tests sont réalisés sur des problèmes 1D, 2D et 3D issus de la discrétisation par différences finies du Laplacien. Les tailles de maillage et les paramètres sont fixés dans `main.py` :

TABLE 1 – Paramètres des expériences (maillages et tolérances)

Dimension	n	Nombre d'inconnues N	k	Tolérance ε	$I_{\max}$	Relaxation
1D	100	100	10.0	10^{-8}	10^4	1.0
2D	50	2500	0.0	10^{-8}	10^4	1.0
3D	30	27000	0.0	10^{-8}	10^4	1.0

Les matrices obtenues par discrétisation du Laplacien sont très grandes mais creuses : chaque point du maillage n'interagit qu'avec un nombre limité de voisins (3 points en 1D, 5 points en 2D, 7 points en 3D). Pour limiter le coût mémoire et accélérer les calculs, la matrice A est stockée au format **CSR**, sous la forme :

$$A = (\text{lig}, \text{col}, \text{val}).$$

- `val` contient les coefficients non nuls de A , rangés ligne par ligne.
- `col` contient, pour chaque coefficient stocké dans `val`, l'indice de colonne correspondant.
- `lig` est un tableau de taille $N + 1$: `lig[i]` donne l'indice dans `val` où commence la ligne i , et `lig[i+1]` l'indice où elle se termine.

Ainsi, les éléments non nuls de la ligne i sont parcourus pour

$$j \in [\text{lig}[i], \text{lig}[i+1[.$$

Les matrices A et les seconds membres b sont construits dans le fichier `data.py` :

- **Lap1DCSR(n,k)** : matrice tridiagonale issue du schéma 1D.
- **Lap2DCSR(n,k)** : matrice 2D (schéma à 5 points) avec un second membre représentant une source rectangulaire centrée.
- **Lap3DCSR(n,k)** : matrice 3D (schéma à 7 points) avec une source sphérique centrée.

Le calcul dominant dans les méthodes itératives est le produit matrice–vecteur Ax . Dans `algo.py`, il est réalisé par la fonction `prodMatCSR(AA, xx)`, qui parcourt uniquement les coefficients non nuls de chaque ligne :

$$(Ax)_i = \sum_{j \in \mathcal{N}(i)} A_{ij} x_j,$$

où $\mathcal{N}(i)$ désigne l'ensemble des colonnes non nulles de la ligne i .

Quatre méthodes sont comparées, toutes appliquées au même système $Ax = b$:

- **Jacobi** : application de D^{-1} au résidu, avec une diagonale précalculée.
- **Descente de gradient** : pas optimal calculé à chaque itération :

$$\alpha_k = \frac{(r^k, r^k)}{(r^k, Ar^k)}, \quad x^{k+1} = x^k - \alpha_k r^k.$$

- **Gradient conjugué** : directions de recherche A –conjuguées.
- **Gradient conjugué préconditionné** : même principe que CG , mais avec un résidu préconditionné

$$w^k = M^{-1}r^k,$$

où M est choisi comme préconditionneur SSOR.

Les performances sont mesurées selon le nombre d'itérations nécessaires pour atteindre $\|r^k\| \leq \varepsilon \|r^0\|$. Le temps CPU, et la décroissance du résidu relatif $\|r^k\|/\|r^0\|$, représentée en échelle logarithmique.

3.2 Resultats en dimension 1D

Dans cette section, nous analysons les performances des différentes méthodes itératives pour le cas unidimensionnel. Ce cas constitue un premier test simple permettant de valider l'implémentation numérique et de comparer le comportement des algorithmes avant de

passer aux dimensions supérieures.

La résolution du système linéaire issu de la discrétisation du Laplacien 1D conduit à une solution numérique régulière sur l'intervalle $[0, 1]$. La solution obtenue est cohérente avec la nature elliptique du problème et avec les conditions aux limites imposées.

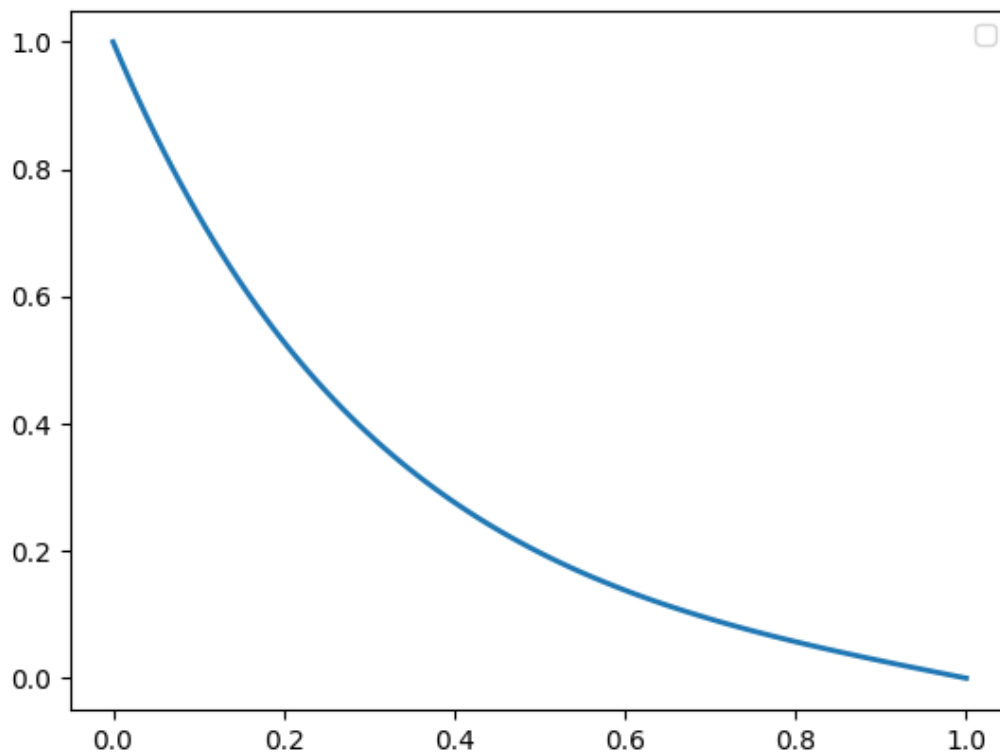


FIGURE 1 – Solution en 1D

La Figure 2 représente l'évolution du logarithme du résidu relatif $\log_{10}(\|r^k\|/\|r^0\|)$ en fonction du nombre d'itérations pour les différentes méthodes étudiées.

On observe des comportements très différents selon la méthode employée :

- les méthodes de Jacobi et de descente de gradient convergent très lentement et atteignent le nombre maximal d'itérations autorisé (10^4) sans satisfaire pleinement le critère d'arrêt ;
- le gradient conjugué (CG) présente une décroissance rapide du résidu, traduisant une convergence efficace ;
- le gradient conjugué préconditionné (PCG) est encore plus performant : le résidu décroît très rapidement dès les premières itérations.

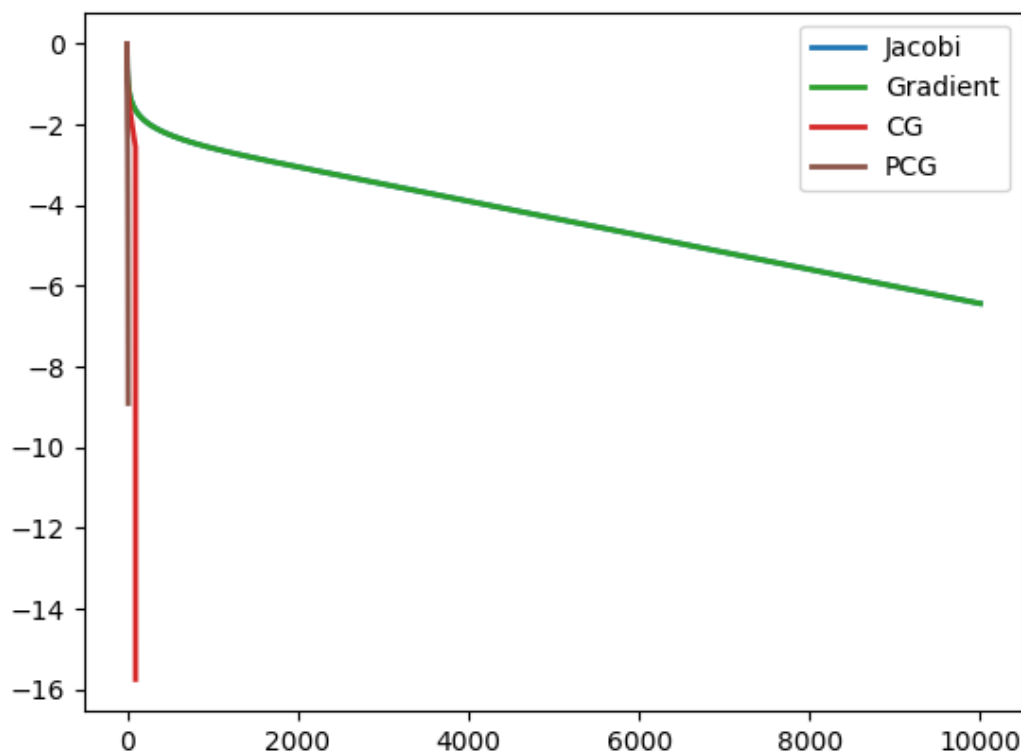


FIGURE 2 – Comparaison de la convergence des méthodes en 1D

Les performances numériques en dimension 1D sont résumées dans le tableau suivant :

TABLE 2 – Comparaison des performances des méthodes en 1D

Méthode	Itérations	Résidu relatif final	Temps de calcul (s)
Jacobi	10000	3.65×10^{-7}	1.67
Gradient	10000	3.65×10^{-7}	2.95
CG	101	1.78×10^{-16}	0.037
PCG (SSOR)	15	1.23×10^{-9}	0.014

En dimension 1D, les méthodes de Jacobi et de descente de gradient sont pénalisées par leur lenteur de convergence. Malgré un coût par itération relativement faible, le grand nombre d'itérations nécessaires entraîne un temps de calcul non négligeable. À l'inverse, la méthode du gradient conjugué converge en un nombre d'itérations très réduit. L'introduction du préconditionneur SSOR améliore encore significativement les performances : le nombre d'itérations est divisé par un facteur supérieur à 6 par rapport à CG, et le temps

de calcul devient quasi négligeable.

3.3 Résultats en dimension 2D

Dans cette section, nous étudions les performances des différentes méthodes itératives pour le cas bidimensionnel. Par rapport au cas 1D, la taille du système augmente significativement ($N = n^2$), ce qui permet de mieux mettre en évidence l'influence de la dimension sur la convergence et le coût numérique des méthodes.

La résolution du problème en 2D permet d'obtenir un champ de solution associé à une source localisée au centre du domaine. À partir du potentiel calculé, un champ de vecteurs est reconstruit par différences finies, ce qui permet de visualiser la structure du champ et ses lignes de courant.

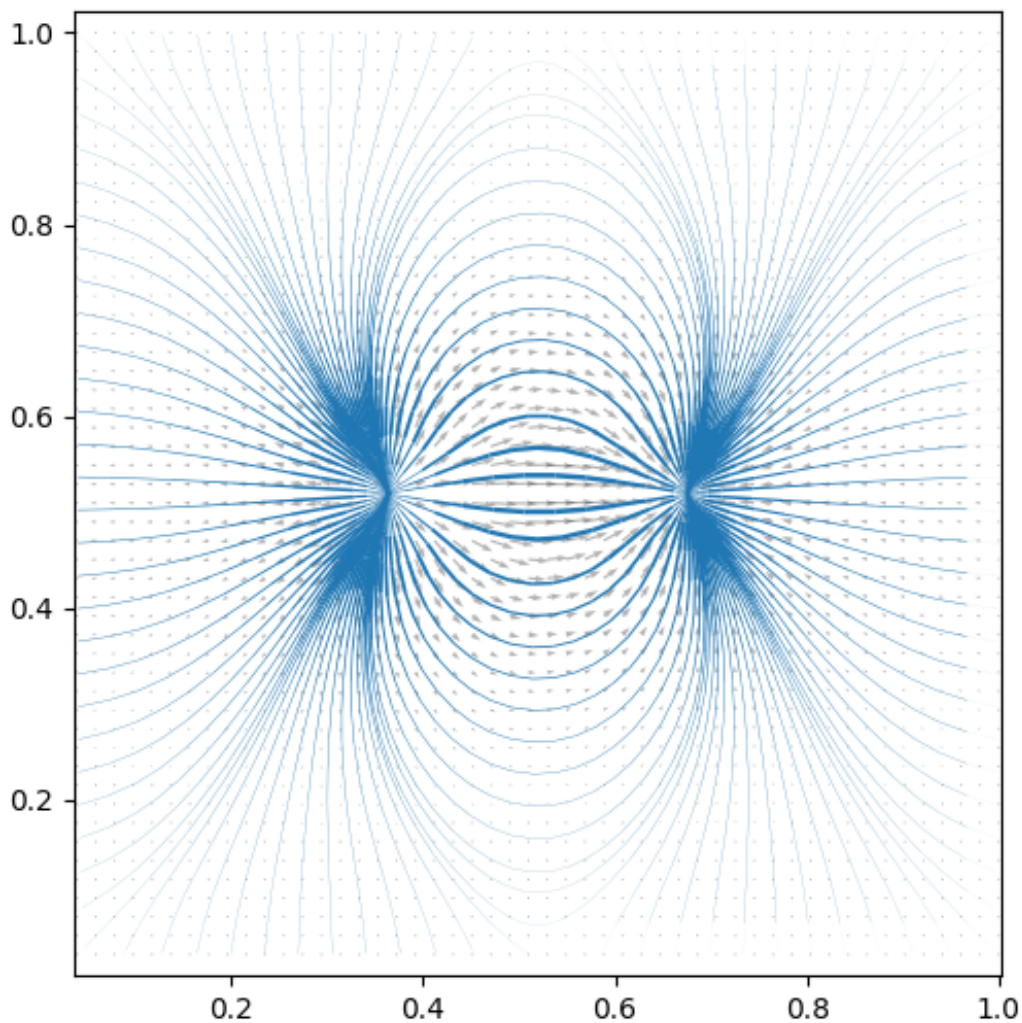


FIGURE 3 – Champ de solution 2D

Cette visualisation montre une structure cohérente avec la source définie dans le second membre, ce qui valide qualitativement la solution numérique obtenue.

La Figure 5 représente l'évolution du logarithme du résidu relatif en fonction du nombre d'itérations pour les différentes méthodes étudiées en 2D.

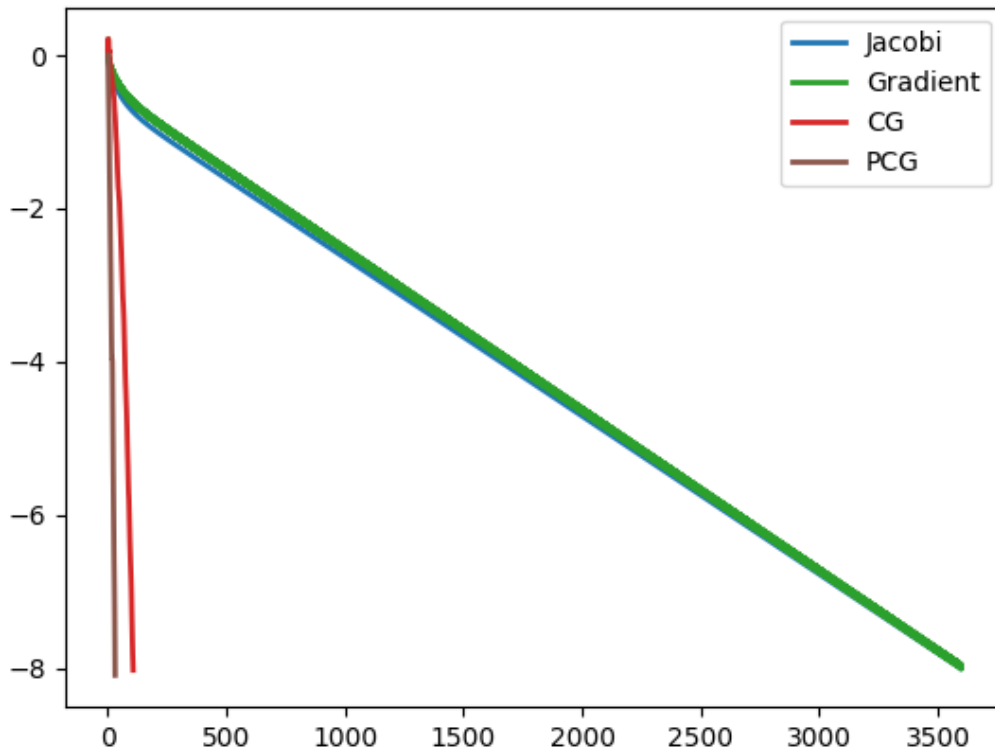


FIGURE 4 – Comparaison de la convergence des méthodes en 2D

On observe que :

- Jacobi et descente de gradient nécessitent plusieurs milliers d'itérations pour atteindre le seuil de convergence, ce qui se traduit par des temps de calcul élevés.
- CG converge beaucoup plus rapidement, avec une décroissance du résidu quasi exponentielle.
- PCG présente la convergence la plus rapide, avec une réduction du nombre d'itérations.

Les performances numériques obtenues en dimension 2D sont regroupées dans le tableau suivant.

TABLE 3 – Performances des méthodes itératives en 2D

Méthode	Itérations	Résidu relatif final	Temps de calcul (s)
Jacobi	3597	9.95×10^{-9}	11.66
Gradient	3599	9.99×10^{-9}	21.99
CG	107	9.45×10^{-9}	0.64
PCG (SSOR)	31	8.12×10^{-9}	0.32

En 2D, l'augmentation du nombre d'inconnues entraîne un coût par itération plus élevé, ce qui pénalise fortement les méthodes nécessitant un grand nombre d'itérations. La méthode de Jacobi et la descente de gradient deviennent rapidement coûteuses, tant en nombre d'itérations qu'en temps de calcul. À l'inverse, le gradient conjugué permet de réduire drastiquement le nombre d'itérations, tandis que l'utilisation du préconditionneur SSOR dans la méthode PCG divise encore ce nombre par un facteur d'environ 3. Cette amélioration se traduit directement par un gain significatif en temps CPU.

3.4 Résultats en dimension 3D

Dans cette section, nous étudions le cas tridimensionnel, qui représente la situation la plus coûteuse du point de vue numérique. Le nombre d'inconnues devient très important ($N = n^3$), ce qui permet d'évaluer la robustesse et l'efficacité des différentes méthodes itératives dans un contexte réaliste.

En dimension 3D, la solution numérique est exploitée pour reconstruire un champ vectoriel (U_x, U_y, U_z) à partir du potentiel calculé par différences finies centrées. Ce champ est ensuite exporté au format VTK et visualisé à l'aide du logiciel ParaView.

La figure met en évidence la structure spatiale du champ généré par la source sphérique définie dans le second membre. Cette visualisation permet de valider qualitativement la cohérence de solution numérique obtenue.

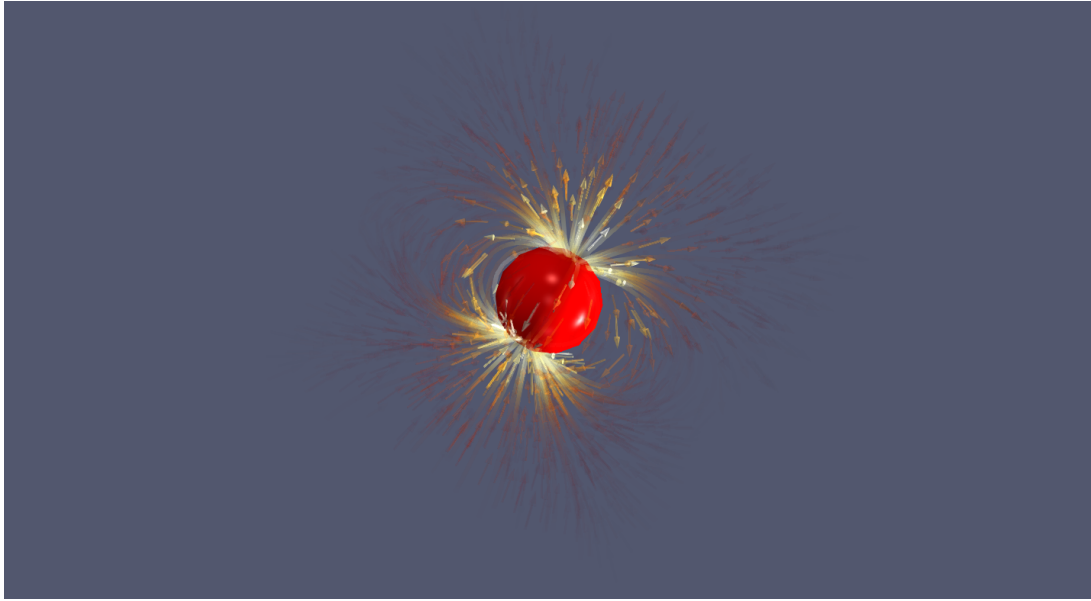


FIGURE 5 – Visualisation du champ vectoriel en 3D (ParaView)

La Figure 6 présente la convergence des différentes méthodes itératives en termes de résidu relatif en fonction du nombre d'itérations.

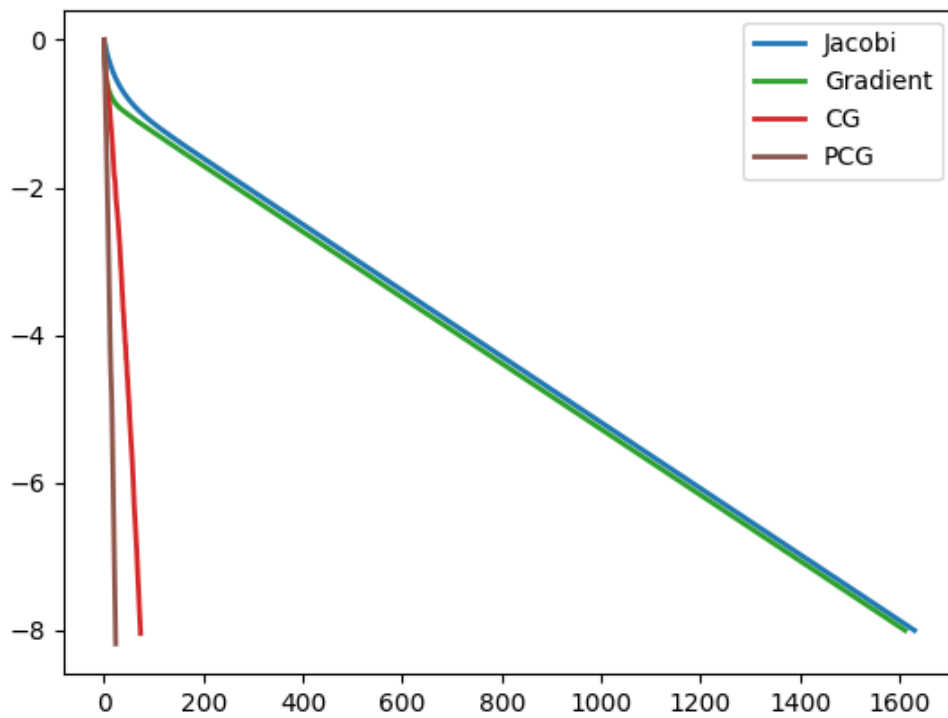


FIGURE 6 – Comparaison de la convergence des méthodes en 3D

On observe que :

- Jacobi et descente de gradient convergent lentement et nécessitent un nombre d'itération élevé pour atteindre le seuil de convergence.
- CG présente une convergence nettement plus rapide.
- PCG reste la méthode la plus performante, avec une décroissance rapide du résidu dès les premières itérations.

Les performances numériques obtenues en dimension 3D sont résumées dans le tableau suivant.

TABLE 4 – Performances des méthodes itératives en 3D

Méthode	Itérations	Résidu relatif final	Temps de calcul (s)
Jacobi	1630	9.99×10^{-9}	63.65
Gradient	1611	1.00×10^{-8}	275.90
CG	74	9.11×10^{-9}	5.44
PCG (SSOR)	24	6.57×10^{-9}	2.84

En dimension 3D, le coût numérique devient particulièrement élevé pour les méthodes nécessitant un grand nombre d'itérations. La descente de gradient est ici clairement inadaptée, avec un temps de calcul supérieur à 270 secondes. À l'inverse, la méthode du gradient conjugué permet de réduire considérablement le nombre d'itérations, tandis que le préconditionnement SSOR dans la méthode PCG divise encore ce nombre par un facteur d'environ 3. Cette réduction se traduit directement par un gain significatif en temps de calcul.

De manière générale, ces résultats confirment les analyses théoriques présentées au Chapitre 3 : pour les systèmes linéaires symétriques définis positifs issus du Laplacien, les méthodes de Krylov sont nettement supérieures aux méthodes itératives classiques. Le préconditionnement joue un rôle clé dans l'efficacité des algorithmes, en particulier lorsque la dimension du problème augmente. Ainsi, pour des problèmes 2D et 3D de taille réaliste, l'utilisation d'une méthode de type PCG avec un préconditionneur adapté est indispensable pour obtenir des performances satisfaisantes.

Conclusion

Dans ce travail, nous avons étudié la résolution numérique de l'équation du Laplacien en dimensions 1D, 2D et 3D à l'aide de différentes méthodes itératives. Après avoir construit le système linéaire issu de la discrétisation par différences finies et l'avoir stocké sous forme creuse au format CSR, plusieurs algorithmes ont été implémentés et comparés : Jacobi, descente de gradient, gradient conjugué et gradient conjugué préconditionné.

Les résultats numériques montrent clairement les limites des méthodes itératives classiques lorsque la dimension du problème augmente. Si les méthodes de Jacobi et de descente de gradient restent applicables en 1D, elles deviennent rapidement inefficaces en 2D et surtout en 3D, en raison d'une convergence lente et de temps de calcul élevés.

À l'inverse, la méthode du gradient conjugué se révèle beaucoup plus efficace pour les matrices symétriques définies positives issues du Laplacien. L'introduction d'un préconditionneur SSOR améliore encore significativement les performances en réduisant à la fois le nombre d'itérations et le temps de calcul, ce qui rend la résolution de problèmes de grande dimension réalisable en pratique. Ce travail met ainsi en évidence le rôle essentiel du préconditionnement dans les méthodes de Krylov et confirme l'intérêt du gradient conjugué préconditionné pour la résolution de systèmes linéaires de grande taille.

Enfin, la résolution du Laplacien est un problème classique qui intervient dans de nombreuses applications physiques. Dans ce travail, son lien avec l'électromagnétisme a été brièvement évoqué, notamment dans la modélisation de champs électromagnétiques stationnaires.

Bilans personnalisés

NGO Tan Minh Duy : À travers ce projet, j'ai pu approfondir ma compréhension du cours d'Analyse numérique en passant de la théorie à la pratique. La mise en œuvre des méthodes itératives m'a permis de mieux saisir des notions parfois abstraites en cours, comme le résidu, le critère d'arrêt ou encore le conditionnement d'une matrice. Le travail sur les différentes méthodes de résolution m'a particulièrement aidé à comprendre les avantages et les limites de chaque approche. Il a ainsi renforcé mon intérêt pour l'Analyse numérique et m'a montré l'importance de ces méthodes dans la résolution de problèmes scientifiques et industriels réels.

NGUYEN Manh Hung : Ce projet m'a permis de mieux comprendre les notions fondamentales abordées en cours d'Analyse numérique, en particulier la résolution de systèmes linéaires issus de la discrétisation d'équations aux dérivées partielles. Le passage du problème continu à un système linéaire discret m'a aidé à faire le lien entre la théorie mathématique et son application numérique concrète. L'étude et l'implémentation des méthodes itératives m'ont permis de mieux appréhender les notions dans le cours.

Références

- [1] A. Tonnoir, *Analyse numérique*, Support de cours, GM3, 2025–2026.
- [2] Wikipédia, *Équation de Poisson*, https://fr.wikipedia.org/wiki/%C3%89quation_de_Poisson, consulté en 2025.
- [3] Wikipédia, *Électrostatique*, <https://fr.wikipedia.org/wiki/%C3%89lectrostatique>, consulté en 2025.
- [4] P. H. Tournier, *Stockage CSR (Compressed Sparse Row)*, CNRS, <https://phtournier.pages.math.cnrs.fr/5mm30/csr/>.
- [5] NumPy Developers, *NumPy Documentation*, <https://numpy.org/doc/>.